An AI buyer that negotiates with suppliers

The hackathon asks us to build an autonomous purchasing agent: software that can negotiate business terms by email, choose a sensible deal, and close it without a person stepping in.

1. What is the game?

We play the buyer for **Hollerau Pumpenwerke**, a fictional German pump maker. Each case is a real-looking purchasing problem: a supplier raises a price, a contract needs renewal, deliveries are late, or several suppliers quote for a new part. The opposing supplier is another AI with its own private pressures and approval rules. We cannot see those private rules.

The buyer receives a **case file** (what the factory needs, its non-negotiable requirements, and its backup option) plus supplier quote emails and sometimes PDFs or spreadsheets. Our agent may send a normal email with up to **three complete alternative term sheets**. The supplier answers; our agent can negotiate again, accept the supplier's current written offer, or walk away. A term sheet is the binding record: promises only in email prose do not make a deal.

> **Simple example:** A supplier offers a part for EUR 10 each but wants payment in 14 days and can cover only half the required quantity. A different package at EUR 10.20 with enough quantity and 60-day payment might be better for the buyer. The agent must compare the *whole deal*, not just the unit price.

2. How does the hackathon judge us?

The Arena estimates the buyer's **total cost over the full need and time horizon**: item price plus effects of uncovered demand, backup buying, payment, delivery, quality, tooling, index clauses, and similar terms. Breaking a buyer must-have scores zero. Otherwise the case score runs from 0 to 1, comparing the deal with the opening quote and a hidden best possible cost. Walking away may be sensible when the backup is better.

There are 19 visible practice case files, but only **10 are hosted for repeated play**. Practice allows 200 new sessions per team per rolling 24 hours; the best score on each hosted case counts. The local simulator is useful for checking the API and message format, but its supplier is scripted and does **not** predict hosted negotiation quality.

For the final, new data is released Sunday morning, code and slides freeze at **12:00**, and the agent plays hidden cases in **five timed waves**. It has about 60 seconds to decide after each supplier reply, and no human may approve each message. The final agent must not read the live results feed or leaderboard.

Primary rules: brief/brief.md, brief/api.md, brief/submission.md. All company names, cases, and supplier records in the package are synthetic.

From quote to decision

Our repository contains the Arena connection, test simulator, purchasing data, several agent strategies, and a newer combined strategy called **unified**.

Stage	In plain language	Where it lives
1. Receive	Open a case, read each supplier's quote, fetch attachments, and remember the conversation.	arena_kit/client.py, play.py, docs.py
2. Understand	Extract supplier facts from its emails, PDF and CSV; keep the source attached. Mark possible motives such as cash or capacity as *likely* only when an explicit statement supports them. Otherwise mark them *unknown*.	supplier_evidence.py, intelligence.py, supplier_profile.py
3. Calculate	Complete the terms, check the buyer's must-haves, and estimate full buyer cost against the backup option. This is a calculator, not a prediction of what the supplier will accept.	arena_kit/cost.py, negotiator/evaluator.py
4. Propose	Generate up to three valid packages. Usually ask for a price improvement. If there is sourced evidence, pair a softer price ask with earlier payment or a larger commitment. Do not invent a supplier preference.	packages.py, package_policy.py
5. Respond	The unified agent chooses a counteroffer, a question, acceptance, or walking away. It compares suppliers, watches remaining moves, avoids repeating an unproductive ask, and closes before a deadline.	arena_kit/agents/unified.py, play.py

What information can it use?

The provided data package has purchasing records, prices, market indices, supplier scorecards, contracts, and hundreds of notes and emails. It is intentionally messy: units may be per item, per 100 or per 1,000; quotes may omit terms; data can contradict a supplier's current story. Claims to the Arena should cite the provided data IDs. The **unified** agent currently works mainly from the live case and supplier thread; it does not automatically reason over the entire historical package.

> **Important distinction:** A term package is our *proposal*, not an accepted deal. The supplier's structured standing offer is what we can accept. A good buyer-cost calculation alone cannot tell us whether a supplier will agree.

Project sources: README.md, STARTER_KIT.md, MVP_EVALUATOR.md, MVP_INTELLIGENCE.md; code paths named above. The separate src/email_negotiation_agent.py is a human-approval email-drafting prototype, not the autonomous Arena path.

PLAN VERSUS CURRENT REALITY

What is built - and what remains

The planned architecture is substantially connected, but a passing local run is not yet evidence that the combined agent negotiates well in the real Arena.

We wanted	What exists now	What is still unproven or missing
Know the buyer's real cost and hard limits	**Built.** Deterministic total-cost evaluator; checks complete sheets, must-haves and fallback cost.	Calibrating decisions on varied hosted cases, not just local arithmetic.
Know the supplier without making up motives	**Built, narrowly.** Supplier-only evidence with source links; explicit-statement profile for cash, capacity, quality and price rigidity.	Many real phrasings or other languages stay unknown; quality has no concrete trade mapping yet. The whole historical corpus is not automatically fed to unified.
Offer buyer-safe give/get packages	**Built.** Up to three complete, costed offers; conservative price ladder for unknown profiles; bounded supplier cost-claim adjustment; cash/capacity trades only with evidence.	Whether those offers are attractive to hosted suppliers. New term combinations still need replay-driven tuning.
Negotiate across rounds and suppliers	**Built.** Unified can counter, ask, accept, or walk; older supplier, packages and advanced agents remain available for comparison.	More hosted regression checks across case types and multi-supplier situations. Unified is **selectable, not the default**.
Be ready for the timed final	**Runner exists.** Parallel case play, deadline-aware closing and Sunday data download are implemented.	Prove the exact chosen agent and release-data workflow end to end before the freeze; prepare submission and slides.

What have tests actually shown?

On this checkout, the automated suite passes: **134 passed, 1 skipped, 15 subtests passed** (the skipped test needs hosted Arena credentials). Earlier practice replay notes show useful lessons and some nonzero scores, but they are not a current scorecard for today's unified code. The repository's quality-gate and practice-score-log documents explicitly describe older versions. The next honest proof is fresh hosted practice with **unified**, checking both the score and the full replay.

Next, in the right order

1. Run a small set of hosted cases with unified and a baseline; save scores and replay links by code version.
2. Read failures: did we misunderstand a term, miss supplier evidence, send an unattractive trade, or accept too soon?
3. Fix the highest-impact pattern, rerun the same cases, then test the Sunday release and final runner.

Status based on this working tree on 10 October 2026, including uncommitted local changes. Plan: TODO.md. Practice lessons: brief/replay-findings-tuning.md. Older evidence: QUALITY_GATE.md and PRACTICE_SCORE_LOG.md. This is an implementation and evidence summary, not a claim of final readiness.